

L09 From NeRF to Gaussian splatting SLAM

A map you can take a photograph of; then the same map, fast; then the same map, built while the camera moves.

From NeRF to Gaussian splatting SLAM

L09 · Unit 1

Three steps today. Each one fixes the problem the last one left.

1. **NeRF**: a map that can render a photograph from any viewpoint. It is slow.
2. **3D Gaussian splatting**: the same rendering idea, hundreds of times faster. It needs the camera poses in advance.
3. **Gaussian Splatting SLAM**: build the splat map and find the poses at the same time, live.

At the end, one slide on SplaTAM, a second splatting SLAM system you will see cited.

Why a robot wants a map that renders

- Voxelblox (L07) answers **is this point free?** That is what a planner needs.
- A rendering map answers **what would the camera see from here?**
- If you can answer that, you can check a guess of where the camera is: render from the guess, compare with the real image, and adjust the guess until they agree.

Render, compare, adjust. That loop is the whole lecture, applied first to the map and then to the camera.

NeRF: a scene as a function

Mildenhall et al., 2020

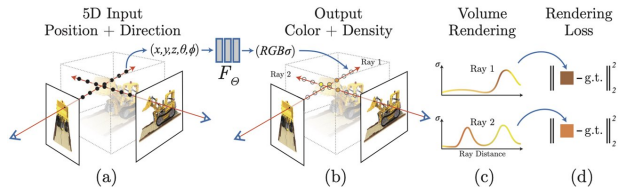


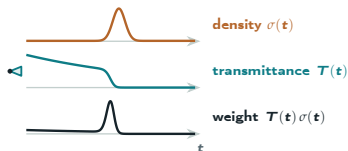
Figure: Mildenhall et al., NeRF repository, MIT License

- A neural network takes a 3D point and a viewing direction, and returns a **colour** and a **density**: how much stuff is at that point.
- Empty air has density near zero. The surface of the bulldozer has high density.
- The whole scene lives in the network's weights. There is no list of points or voxels.

Rendering one pixel

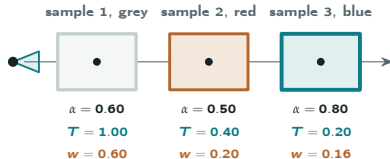
1. Shoot a ray from the camera through the pixel.
2. Pick sample points along the ray.
3. Ask the network for the colour and density at each.
4. Walk from front to back. Each sample takes its share of the light that has **not yet been blocked**.

Density turns into an opacity α between 0 and 1 for each sample: the fraction of the remaining light it stops.



Blending, by numbers

the one calculation today



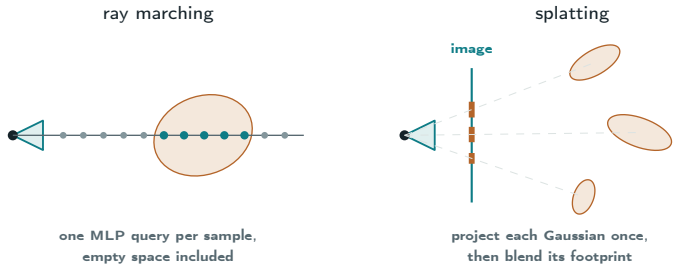
■ Opacities $\alpha = (0.6, 0.5, 0.8)$. What weight does each sample get in the pixel?

Training a NeRF

- Render a pixel, compare it with the real photo, and nudge the network's weights by gradient descent (L05). Every step of the blending is differentiable.
- Repeat for millions of pixels from dozens of photos.
- The photos must come with **camera poses**. NeRF gets them from COLMAP (L04), run first.

Render, compare, adjust: here the thing being adjusted is the network.

What NeRF costs



- NeRF asks its network 256 times per pixel. An 800×800 image costs about 164 million network evaluations, most of them in empty air.
- Training takes hours to days; one image takes seconds. Too slow to track a camera at 30 frames per second.

Gaussian splatting: keep the blend, drop the network

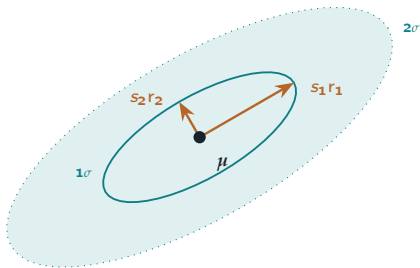
Kerbl et al., SIGGRAPH 2023

- Store the scene as an explicit list of millions of small, fuzzy, coloured blobs: **3D Gaussians**.
- To render, project each blob onto the image once, sort by depth, and blend with the same front-to-back rule.
- No network, and no samples in empty space: where there is no blob, there is no work.



Figure: Kerbl et al., 3D Gaussian Splatting (SIGGRAPH 2023), CC BY 4.0

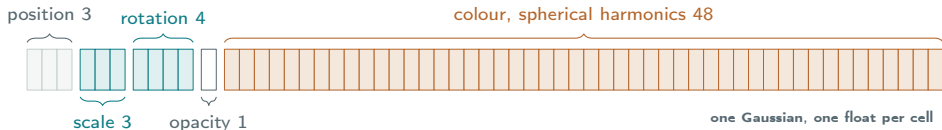
One Gaussian



- **Centre μ :** where it sits.
- **Shape:** an ellipsoid, given by a rotation and three lengths s_1, s_2, s_3 . Covariance Σ packs these into one matrix.
- **Opacity α :** how much light it blocks at its centre.
- **Colour:** can change a little with viewing direction, for shine.

It is solid at the centre and fades out smoothly; there is no hard edge.

A scene is millions of them

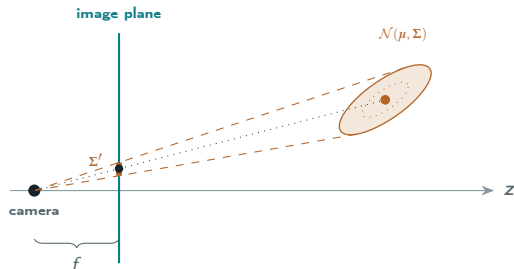


■ Numbers per Gaussian, and how many fit in the paper's average model

- Most of the numbers are colour: 48 of 59, so that colour can vary with direction.
- The SLAM systems later today keep only the 3 base colour numbers.

Drawing an image

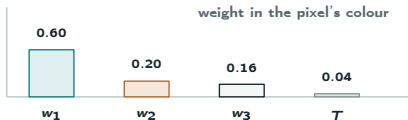
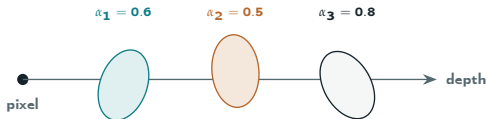
project, sort, blend



1. **Project** each Gaussian through the pinhole camera (L01). The centre lands on a pixel; the 3D ellipsoid lands as a 2D ellipse, its **footprint**.
2. **Sort** the Gaussians by depth.
3. **Blend** front to back at each pixel, exactly as on the NeRF slide.

Footprint size shrinks with distance: a blob of radius 0.05 m at 2 m, with $f = 500$ px, is about $f\sigma/Z = 12.5$ px across one standard deviation.

Blending splats: the same sum



Three splats cover this pixel with the same three opacities as before, so they get the same weights: 0.60, 0.20, 0.16.

Where each α_i comes from	
NeRF	a network query at a sample on the ray
Splatting	a Gaussian's opacity times its footprint at the pixel

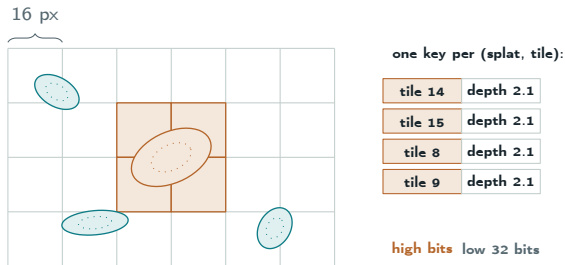
Same blending rule. Only the source of the α values changed.

Depth and coverage for free

- Blend each splat's **depth** instead of its colour, and you get a depth image. For the three splats at 1.8, 2.1 and 2.6 m: $D = 1.92$ m.
- Blend the number **1** instead, and you get how much of the pixel the map covers: $S = 0.96$. Call it the **silhouette**.
- S near 1: the map explains this pixel. S near 0: the map has nothing here yet.

Both come back in SLAM: depth to compare with a depth camera, the silhouette to know what is mapped.

Why it is fast



- Cut the image into 16×16 pixel tiles, and list which Gaussians touch each tile.
- Sort everything by tile, then by depth, in **one** GPU sort per image.
- Each pixel walks its tile's short list and blends. This is the kind of work GPUs were built for.

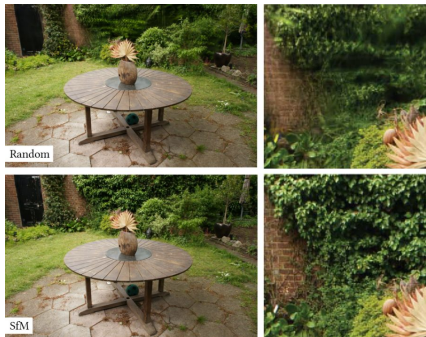
Fitting a scene



- Render a training view, compare with its photo, and push gradients back to every Gaussian's position, shape, opacity and colour.
- The comparison is mostly the average pixel error, plus a term that rewards matching local texture (SSIM).

Render, compare, adjust: now the thing being adjusted is the list of Gaussians.

Where the Gaussians start



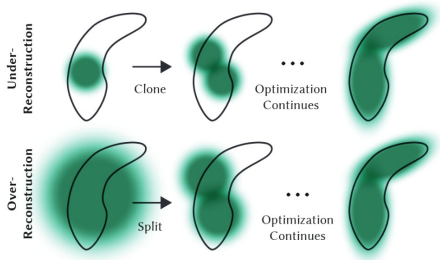
- COLMAP (L04) gives the camera poses and a sparse 3D point cloud.
- Each point becomes one small round Gaussian with the point's colour.
- Starting from random points also works, but the background comes out blurred (top).

Offline splatting needs the camera poses before it starts.

Figure: Kerbl et al., 3D Gaussian Splatting (SIGGRAPH 2023), CC

BY 4.0

Adding and removing Gaussians



Every 100 steps, look for Gaussians the optimiser keeps trying to move: that region is not explained well.

- **Clone** a small one to fill a gap (top).
- **Split** a large one that covers too much (bottom).
- **Prune** ones that have become nearly transparent.

Figure: Kerbl et al., 3D Gaussian Splatting (SIGGRAPH 2023), CC BY 4.0

The whole pipeline

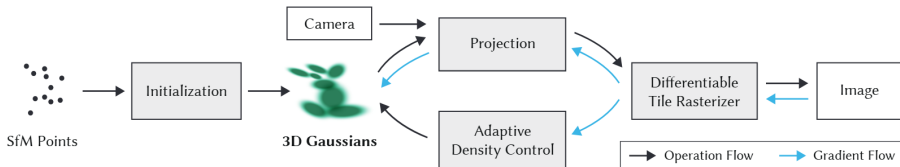
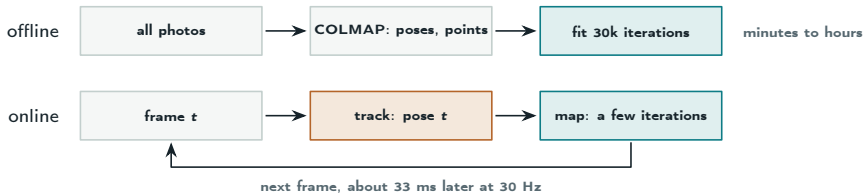


Figure: Kerbl et al., 3D Gaussian Splatting (SIGGRAPH 2023), CC BY 4.0

- SfM points become Gaussians; project and rasterise into an image; compare; send gradients back (blue arrows); add and remove Gaussians.
- Result: real-time rendering at 1080p, after under an hour of training.

Mip-NeRF360 scenes	Train	fps	PSNR
Mip-NeRF360	48 h	0.06	27.69
Instant-NGP (big)	7.5 min	9.4	25.59
3DGS, 30K iterations	42 min	134	27.21

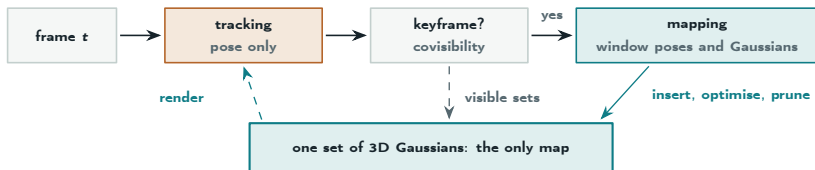
From offline fitting to SLAM



- **No poses:** each new frame's pose must be found against the map being built.
- **No SfM points:** Gaussians must be added from the frames themselves.
- **A time budget:** one frame every 33 ms. Only a few optimisation steps per frame.

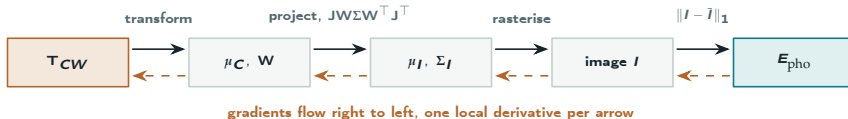
Gaussian Splatting SLAM

Matsuki, Murai, Kelly and Davison, CVPR 2024



- One set of Gaussians does everything: tracking, choosing keyframes, mapping, rendering.
- Works with a single RGB camera, and uses depth when an RGB-D camera provides it.
- Runs live at about 3 frames per second from a monocular camera on a desktop GPU.

Tracking: render, compare, nudge the pose

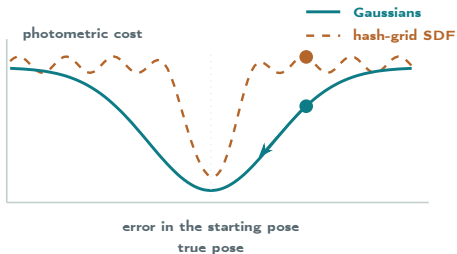


$$T^* = \arg \min_T \left\| \text{render}(\text{Gaussians}, T) - \text{image} \right\|_1$$

- Freeze the Gaussians. Render from a guess of the camera pose T , compare with the new image, and step the pose downhill.
- Like PnP (L03), but every pixel is a residual instead of a few matched features.

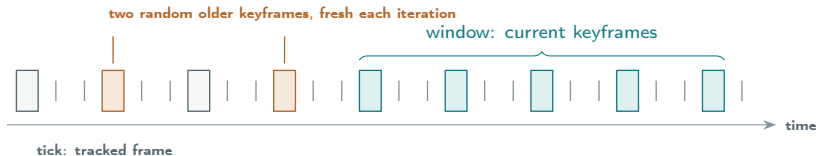
Render, compare, adjust: now the thing being adjusted is the camera.

Why Gaussians are easy to track against



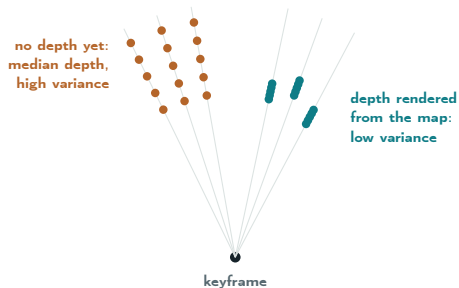
- Gradient descent only finds the true pose if it starts inside the bowl around it.
- Each Gaussian's footprint is smooth and several pixels wide, so the error changes smoothly as the camera moves: a wide bowl.
- From random starting errors, about **80%** of tracking runs succeed on a Gaussian map, against 14% on a hash-grid neural map.

Keyframes and a mapping window



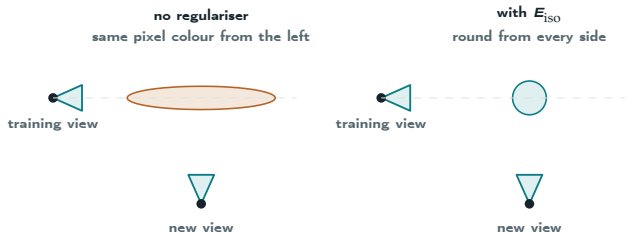
- A frame becomes a **keyframe** when it sees a noticeably different set of Gaussians from the last keyframe, or the camera has moved far.
- **Mapping** keeps a window of about 10 recent keyframes and adjusts the Gaussians **and** those keyframe poses together: bundle adjustment (L04) with splats in place of points.
- It also revisits a couple of random older keyframes, so it does not overwrite what it can no longer see.

Adding Gaussians as you go



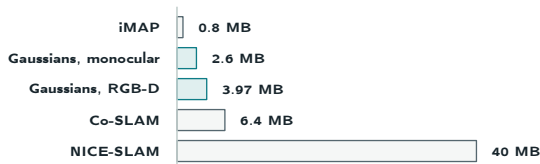
- At each keyframe, add Gaussians where the map has nothing yet.
- **With a depth camera:** place them at the measured depth, as in L01.
- **With one RGB camera:** guess the depth from the map, or around the image's median depth.
- Wrong guesses never agree with later views, so they are pruned.

Keeping Gaussians round



- A Gaussian stretched along the viewing ray looks the same from that view, so the image loss cannot see the stretch.
- A later view then sees a long streak.
- Fix: a small penalty for each Gaussian's three lengths differing from their average.

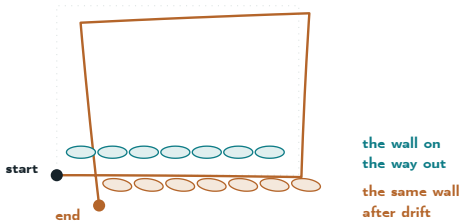
What it achieves



Map memory on TUM RGB-D, in MB.

- A desk-sized map in a few megabytes.
- Renders new views at hundreds of frames per second, against under 1 for NICE-SLAM (L07).
- Tracking as accurate as the best dense SLAM systems without loop closure, from one camera.

What is still hard



- **Drift:** small tracking errors add up. Nothing ties the pose now to where the camera was ten minutes ago.
- **Loop closure:** coming back to a place after drift puts a second copy of it in the map.
- A pose graph (L04) can fix the keyframe poses quickly, but millions of Gaussians do not move with them.

SplaTAM, briefly

Keetha et al., CVPR 2024

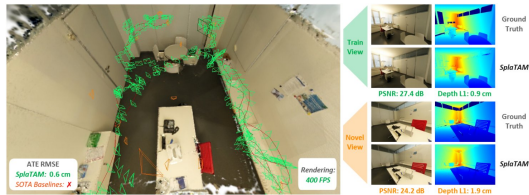


Figure: Keetha et al., SplaTAM (CVPR 2024), CC BY-SA 4.0

- Another splatting SLAM system, from the same year. RGB-D only.
- Simpler Gaussians: round, one colour, 8 numbers each.
- Leans on the **silhouette**: track only on pixels the map already covers, and add Gaussians where it does not.

SplaTAM and Gaussian Splatting SLAM, side by side

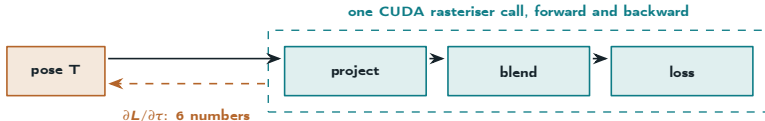
	SplaTAM	Gaussian Splatting SLAM
Camera	RGB-D only	one RGB camera, or RGB-D
Gaussians	round, one colour: 8 numbers	ellipsoids, nudged toward round
Tracking	only pixels the map covers ($S > 0.99$)	all pixels
New Gaussians	where the silhouette is empty	at keyframes, from depth or a guess
Mapping	Gaussians only; poses stay fixed	Gaussians and keyframe poses together
Speed	about 2.4 s per frame	about 3 frames per second

Same: one Gaussian map, rendered and compared with each frame to find the pose and then to refine the map. Neither closes loops.

SplaTAM keeps it simple and leans on depth. Gaussian Splatting SLAM does more work so one camera is enough.

Why Gaussian Splatting SLAM tracks faster

1 of 2: the pose gradient, inside the rasteriser



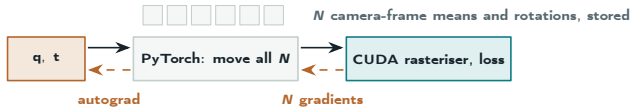
MonoGS, the code of Gaussian Splatting SLAM, writes out the derivative of the rendered image with respect to the camera pose on $SE(3)$, by the chain rule:

$$\frac{\partial I}{\partial \tau} = \underbrace{\frac{\partial I}{\partial \mu_I}}_{\text{blend}} \underbrace{\prod}_{\text{pinhole}} \underbrace{[I - \mu_C^\wedge]}_{\text{nudge the pose}}$$

Every factor is known in closed form, so it is computed inside the rasteriser's backward pass. Each tracking iteration is **one fused forward and backward pass**.

Why Gaussian Splatting SLAM tracks faster

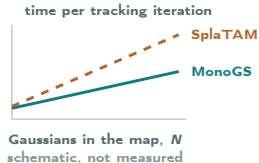
2 of 2: *SplaTAM's pose goes through autograd*



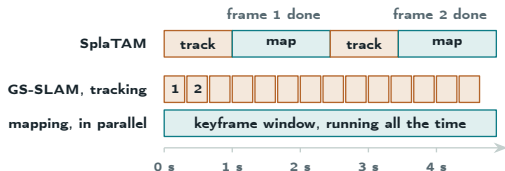
SplaTAM keeps the pose as q, t in PyTorch. Every iteration it:

1. moves all N Gaussians into the camera frame with PyTorch operations, keeping the results;
2. lets autograd backpropagate through that transform, for every Gaussian.

More memory traffic, more kernel launches, and a cost that **grows with the map**.



The rest of the saving



- **Mapping leaves the frame.** Gaussian Splatting SLAM maps its keyframe window in a parallel process; a frame waits only for tracking. SplaTAM tracks, then maps, every frame.
- **Fewer Gaussians, cheaper iterations.** SplaTAM adds one per uncovered pixel, every frame (307{,}200 on the first). Gaussian Splatting SLAM adds them only at keyframes.
- **Fewer tracking iterations.** Gaussian Splatting SLAM stops once the pose stops changing; SplaTAM runs a fixed count.

Splat map or TSDF on a robot?

	Voxblox (L07)	Gaussian splat map
Renders photos	no	yes, fast
Free space	stored: voxels seen empty	not stored
Tracking	needs separate odometry	against the map itself
Compute	CPU	GPU

A splat map answers what the scene looks like. A planner asks where it is safe to go.

What you should be able to do

- Explain how NeRF renders a pixel, and why it is slow.
- Compute blending weights for a few opacities along a ray.
- Say what one Gaussian stores, and how splatting draws an image.
- Explain why offline splatting needs poses, and how Gaussian Splatting SLAM finds them.
- Say what SplatAM uses its silhouette for.
- Decide whether a robot task needs a splat map, a TSDF, or both.

Next

L10: feed-forward 3D foundation models

Every map this unit has been fitted to one scene, by optimisation, over minutes. Wednesday's networks predict geometry and camera poses for a new scene in one forward pass.

- **Responses due Sun Oct 11:** today's, on Gaussian Splatting SLAM, and Wednesday's, on DUS_t3R (Wang et al.).
- **Also assigned:** MAST₃R (Leroy et al.) and VGGT (Wang et al.).
- Coming with them: attention and transformers, deferred from Unit 0.